VidTalk Application Architecture

1. Project Overview

VidTalk is a native Android video-sharing and discussion application where users can watch videos and interact through text and short video comments.

The key feature is timestamp-based commenting, where users can attach comments to a specific point in a video. These timestamps are displayed as markers on the video progress bar, allowing users to jump directly to the relevant discussion.

2. High-Level Architecture

3. Technology Stack

Layer	Technology
Mobile	Kotlin
UI	Jetpack Compose
Architecture	MVVM
Networking	Retrofit + OkHttp
Async Operations	Kotlin Coroutines + Flow
Dependency Injection	Hilt
Local Database	Room
Video Playback	Media3 / ExoPlayer
Camera	CameraX
Backend	Python + FastAPI
ORM	SQLAlchemy 2.x
Database	PostgreSQL
Migrations	Alembic
Authentication	Google OAuth / Google Sign-In
Video Storage	Cloudinary
Version Control	Git + GitHub

4. Android Architecture

The Android application follows MVVM architecture.

Responsibilities

Jetpack Compose

- Displays screens
- Displays videos
- Displays comments
- Handles user interaction

ViewModel

- Holds UI state
- Processes user actions
- Calls repositories

Repository

- Acts as the data-access layer
- Communicates with APIs and local storage

Retrofit

- Communicates with FastAPI

Room

- Provides local Android data storage where required

5. Backend Architecture

The FastAPI backend follows a layered structure.

This separation keeps API handling, business logic, and database operations independent.

6. Database Architecture

PostgreSQL stores the application's structured information.

Initial/core entities:

A comment can contain:

- comment_id
- user_id
- video_id
- text
- timestamp
- parent_comment_id
- created_at

`timestamp` identifies the position in the video.

`parent_comment_id` allows nested replies.

7. Video Architecture

Actual video files are stored in Cloudinary, not PostgreSQL.

```
Android App
      |
      ▼
FastAPI API
      |
      ▼
  Cloudinary
      |
      ▼
Video URL
      |
      ▼
 PostgreSQL
 stores URL
```

For playback:

```
Android
   |
   ▼
FastAPI
   |
   ▼
Video metadata + URL
   |
   ▼
Media3 / ExoPlayer
   |
   ▼
Cloudinary
   |
   ▼
Video playback
```

8. Timestamp Comment Architecture

This is one of VidTalk's main features.

```
Video Timeline

0:00 ──●──────────●──────────●── 10:00
       |          |          |
     Comment    Comment    Comment
     1:32       5:14       8:42
```

When a user creates a comment while watching:

```
Current video position
        |
        ▼
   01:32 seconds
        |
        ▼
Create comment
        |
        ▼
Save timestamp
```

```
        |
        ▼
PostgreSQL
```

When the user clicks the marker:

```
Blue Marker
     |
     ▼
Timestamp = 01:32
     |
     ▼
ExoPlayer.seekTo(01:32)
     |
     ▼
Video jumps to 01:32
```

9. Authentication Flow

```
User
 |
 ▼
Android App
 |
 ▼
Google Sign-In
 |
 ▼
Google Authentication
 |
 ▼
Backend
 |
 ▼
Identify/Create User
 |
 ▼
PostgreSQL
```

The backend is responsible for validating the authenticated user's identity and controlling access to protected API operations.

10. Comment System

VidTalk supports:

```
Video
 |
 ├── Comment
 |    ├── Reply
 |    |    └── Reply
 |    |
 |    ├── Like
 |    └── Dislike
 |
 ├── Comment
```

```
 |
 └── Comment
```

Comments may be:

- Text comments
- Short video comments
- Timestamp-linked comments
- Replies to other comments

11. End-to-End Data Flow

Example: User opens a video

```
User
 ↓
Android Video Screen
 ↓
ViewModel
 ↓
Repository
 ↓
Retrofit
 ↓
GET /videos/{id}
 ↓
FastAPI
 ↓
SQLAlchemy
 ↓
PostgreSQL
 ↓
Video metadata + Cloudinary URL
 ↓
Android
 ↓
Media3 / ExoPlayer
 ↓
Video Playback
```

Example: User posts a comment

```
User
 ↓
Android UI
 ↓
ViewModel
 ↓
Repository
 ↓
Retrofit
 ↓
POST /comments
 ↓
FastAPI
 ↓
Validation
```

```
 ↓
SQLAlchemy
 ↓
PostgreSQL
 ↓
Comment created
 ↓
Response
 ↓
Android UI updated
```

12. Production Deployment Architecture

The planned production architecture is:

GitHub will be used for source-code management and can later be integrated with CI/CD.

13. Architecture Principles

The system is designed around these principles:

- Native Android experience
- Separation of concerns
- REST API communication
- Secure authentication
- Relational database integrity
- Dedicated media storage
- Modular backend
- Scalable architecture
- Maintainable codebase
- Clear separation between mobile, backend, database, and media storage

Client Summary

VidTalk uses a three-tier architecture consisting of a native Android client, a Python FastAPI backend, and a PostgreSQL database. The Android application follows MVVM architecture and communicates with the backend through secure REST

APIs. PostgreSQL manages structured application data, while Cloudinary manages video files. Google OAuth provides user authentication, and Media3/ExoPlayer and CameraX handle video playback and recording on Android.